Naive Bayes

Classification via Bayes' theorem

Supervised Learning · Classification

$$P(\text{class} \mid \text{features}) = \frac{P(\text{features} \mid \text{class})P(\text{class})}{P(\text{features})}$$

generative probability · conditional independence · fast closed-form estimates

Bayes' Theorem for Classification

$$P(y=k \mid x) = \frac{P(x \mid y=k)P(y=k)}{P(x)} \propto P(x \mid y=k)P(y=k)$$

$$\hat{y} = \arg\max_{k} P(x \mid y=k)P(y=k)$$

Prior
`P(y=k)`
class frequency

Likelihood
`P(x | y=k)`
feature model

Posterior
`P(y=k | x)`
class after evidence

Evidence
`P(x)`
same for every k

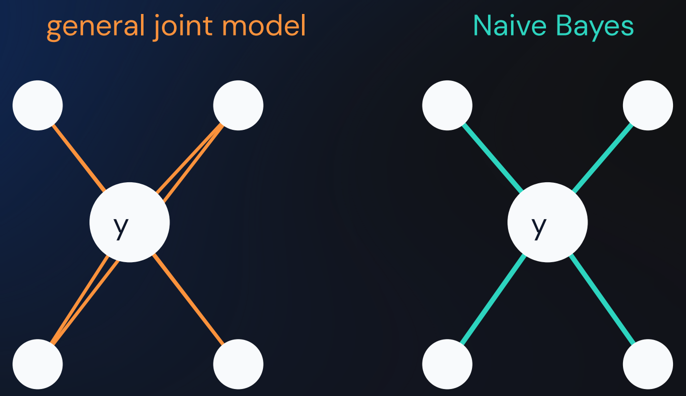
Drop `P(x)` only for the arg-max: it changes normalization, not the winning class.

The “Naive” Assumption

$$P(x_1, \ldots, x_d \mid y = k) \approx \prod_{j=1}^{d} P(x_j \mid y = k)$$

$$\hat{y} = \arg\max_k P(y = k) \prod_j P(x_j \mid y = k)$$

- Assume features are conditionally independent given the class
- Usually false—“New” and “York” clearly co-occur
- Yet class rankings can remain useful even when probabilities are distorted



A Worked Posterior by Hand

Email contains "free" ($x_1{=}1$) and "click" ($x_2{=}1$)

$$P(\text{spam}) = .4, \quad P(\text{ham}) = .6$$

$$P(x_1{=}1 \mid \text{spam}) = .7,\ P(x_1{=}1 \mid \text{ham}) = .1$$

$$P(x_2{=}1 \mid \text{spam}) = .6,\ P(x_2{=}1 \mid \text{ham}) = .2$$

- spam score $= .4 \times .7 \times .6 = .168$
- ham score $= .6 \times .1 \times .2 = .012$
- normalize: $.168 + .012 = .18$
- $P(\text{spam} \mid x) = .168/.18 \approx .933$

```
p_spam, p_ham = 0.4, 0.6
lik_spam = 0.7 * 0.6      # P(x1|spam) * P(x2|spam)
lik_ham = 0.1 * 0.2       # P(x1|ham)  * P(x2|ham)

score_spam = p_spam * lik_spam   # 0.168
score_ham = p_ham * lik_ham      # 0.012

total = score_spam + score_ham   # 0.18
posterior_spam = score_spam / total
assert round(posterior_spam, 3) == 0.933
```

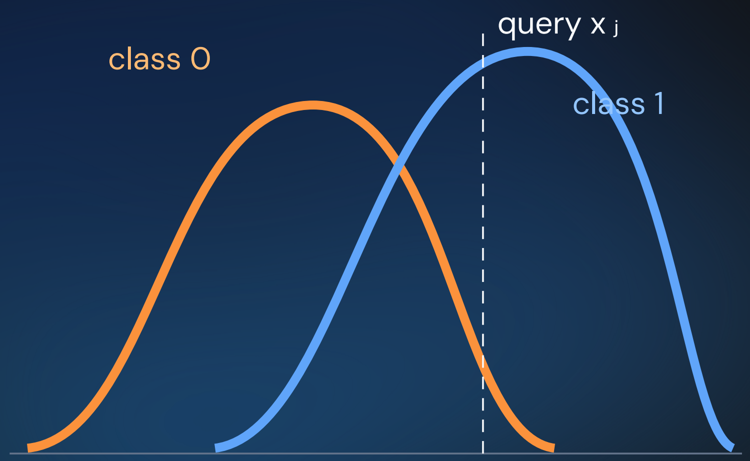
Gaussian Naive Bayes · Continuous Features

$$P(x_j \mid y = k) = \frac{1}{\sqrt{2\pi\sigma_{jk}^2}} \exp\left[-\frac{(x_j - \mu_{jk})^2}{2\sigma_{jk}^2}\right]$$

$$\hat{\mu}_{jk} = \frac{1}{n_k} \sum_{i:y_i=k} x_{ij}, \qquad \hat{\sigma}_{jk}^2 = \frac{1}{n_k} \sum_{i:y_i=k} (x_{ij} - \hat{\mu}_{jk})^2$$

```python
from sklearn.datasets import make_classification
from sklearn.naive_bayes import GaussianNB

X, y = make_classification(
    n_samples=200, n_features=4,
    n_informative=3, n_redundant=0,
    random_state=0)
model = GaussianNB().fit(X, y)
print(model.theta_, model.var_)
print(model.predict_proba(X[:3]))
```



Counts, Binary Features, and Smoothing

Multinomial NB
word or event counts

Bernoulli NB
presence / absence

$$P(x_j = v \mid y = k) = \frac{\text{count}(x_j = v, y = k) + \alpha}{\text{count}(y = k) + \alpha|V|}$$

Without smoothing: one unseen word gives probability 0, collapsing the entire product.

```
from sklearn.feature_extraction import text
from sklearn.naive_bayes import MultinomialNB

docs = ["free money click", "project meeting",
        "win free prize", "project deadline"]
y = [1, 0, 1, 0]
vectorizer = text.CountVectorizer()
X = vectorizer.fit_transform(docs)
model = MultinomialNB(alpha=1).fit(X, y)
new = vectorizer.transform(["free prize click"])
print(model.predict(new), model.predict_proba(new))
```

Naive Bayes vs. Logistic Regression

	Naive Bayes	Logistic Regression
Models	`P(x | y)` then Bayes	`P(y | x)` directly
Type	generative	discriminative
Fit	closed-form statistics	iterative optimization
Thrives on	small, sparse, high-d data	correlated predictive features
Risk	independence distorts probabilities	more data and fitting time

Under certain Gaussian assumptions, both converge to the same boundary with abundant data—but reach it differently and behave differently when data is scarce.

Naive Bayes in One View

Invert

likelihood × prior becomes posterior ranking

Factor

conditional independence makes the joint tractable

Match the variant

Gaussian, Multinomial, or Bernoulli

Smooth

avoid zero-probability collapse

Next: **Decision Trees** — interpretable if/else rules with no probability model or distance metric.